Software Testing (CSA37)

ASSIGNMENT 1

CO1: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository, defect examples.

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as 500 on the booking page but 550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect	Remarks

			( )	
Requirements	**30**	**2**	**500**	**No formal review**
Design	**22**	**3**	**1,500**	**No formal review**
Coding	**38**	**15**	**3,000**	**Developer self-check only**
Testing	**5**	**20**	**8,000**	**Demo-based testing**
Documentation	**5**	**0**	**–**	**–**
Production (after release)	**–**	**60**	**40,000**	**Reported by patients**

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected

A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- **The role of a defined process in achieving software quality**
- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

Your position must be defended with evidence from the case.

Evidence Expected

A justified position on the role of process in quality, supported by the case data.

TASK 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected

Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

TASK 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- **At least four software testing principles that apply to the MediQueue situation**

For each principle, the incident or decision in the case that violates or confirms it

Which principle, if followed, would have had the largest effect on quality, and why

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected

A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

TASK 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

The role and responsibilities of a tester across the life cycle

The advantages and limitations of developers testing their own code

Which incidents a tester would most likely have caught, and at which stage

The qualities a tester for a healthcare application must have

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

TASK 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- **The origin of each incident: requirements, design, code, testing, documentation or environment**
- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**
- **What the overall pattern of origins tells the hospital about where to invest effort**

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**

The class, severity and priority of each incident, with reasons

Why severity and priority may differ for the same defect (for example, I7 and I1)

Which defect class the team should target first, and why

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

TASK 9 – Design and Appraise a Defect Repository 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

The fields a defect repository for MediQueue must contain, and why each is needed

How the repository supports defect analysis, process improvement and prevention

The weaknesses of the current e-mail and spreadsheet approach

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- **The process changes you recommend and their order of priority**
- **The role of testers, reviews and the defect repository in your plan**

The expected effect on the number and cost of defects

Trade-offs, risks and limitations of your plan

Whether the plan remains practical for a small team of five developers

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- **Whether the planned reviews and test activities fit into a two-week cycle**
- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **The defect data and cost figures**
- **Defect origins and classes**
- **Release frequency**
- **Patient-safety and data-privacy risk**
- **Team size and capacity**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

TASK 1 – EVALUATE TESTING AS AN ENGINEERING ACTIVITY

MediQueue testing cannot be considered a proper engineering activity. Engineering quality requires planning, measurement, standards and repeatability. In this project, the five developers tested their own code just before release, and management approved the application after a demonstration. This was not enough to prove that the complete system was reliable.

Planning means deciding what to test, how to test it, who will test it and when testing should take place. MediQueue had no separate tester or clear test plan. Therefore, important situations such as two patients booking at the same time (I1) and two doctors saving prescriptions together (I6) were not properly checked.

Measurement means recording test coverage, test results, defects and progress. The team depended on a demonstration instead of measurable release conditions. The defect data shows that 60 defects were reported after release, which indicates that the testing evidence was not sufficient.

Standards mean following agreed rules for requirements, coding, reviews and testing. There was no formal review of requirements or design. This contributed to the missing Sunday rule (I2) and misunderstanding of the GST rule (I3).

Repeatability means that another tester should be able to perform the same test using written steps and get consistent results. MediQueue mainly used a demo-based approach. Boundary cases such as names longer than 40 characters (I4) and patients turning 60 on the appointment date (I5) could be missed. Incident I8 also shows that a test can be repeated but still be wrong if its expected result is incorrect.

An engineered approach would include a written test plan, requirement and design reviews, test cases with correct expected results, a suitable test environment, independent testing, defect recording and clear release criteria. Therefore, MediQueue only hoped for quality instead of engineering it into the product.

TASK 2 – EVALUATE THE ROLE OF PROCESS IN SOFTWARE QUALITY

Software quality depends on both skilled people and a good process. Skilled developers can write good code, but a defined process makes sure that requirements are understood, designs are reviewed, code is checked and the complete application is tested.

Better developers alone would not have prevented all MediQueue incidents. For example, the Sunday closure rule was never written in the requirements (I2), so developers could not know it unless they asked the hospital staff. The GST rule was misunderstood (I3), which required clarification from the billing department. Incidents I1 and I6 needed design reviews and tests for simultaneous activities. I8 needed an independent check of the expected refund value.

The defect data shows that 30 defects were introduced in requirements and 22 in design, making 52 defects in total. This proves that problems began before coding. Therefore, improving only coding skills will not solve the quality problem.

The team is closest to TMM Level 1 – Initial because testing was informal, depended on developers and happened mainly before release. The team should move towards Level 2 by defining testing responsibilities, basic plans, test techniques and release conditions. A good process is therefore more important than depending only on individual skill.

TASK 3 – EVALUATE TESTING AS A PROCESS

Testing should be carried out throughout the software development life cycle and not only just before release. The main testing activities are planning, test design, environment preparation, execution, evaluation and reporting.

During planning, the team identifies risks, testing scope, people and schedule. During test design, test cases and expected results are prepared. The test environment is then arranged with sample patients, doctors and payment and SMS services. During execution, tests are run and results are recorded. During evaluation, actual results are compared with expected results. Finally, defects and test progress are reported.

Verification checks whether each development output is prepared correctly. Validation checks whether the final application meets the hospital’s needs. Requirements should be reviewed

with hospital staff to detect I2 and I3. Design reviews should check token generation and shared patient data to prevent I1 and I6. Coding reviews and unit tests should check name length and age calculation for I4 and I5. System and integration testing should check payments, SMS and performance for I3, I9 and I10. Documentation review should detect I7. Test-case review should detect the incorrect expected value in I8.

The cost of fixing defects is calculated by multiplying the number of defects detected in each phase by the cost per defect.

Phase	**Defects detected**	**Cost per defect**	**Total cost**
Requirements	2	₹500	₹1,000
Design	3	₹1,500	₹4,500
Coding	15	₹3,000	₹45,000
Testing	20	₹8,000	₹1,60,000
Production	60	₹40,000	₹24,00,000
TOTAL	100	—	₹26,10,500

Total cost = (2 × 500) + (3 × 1,500) + (15 × 3,000) + (20 × 8,000) + (60 × 40,000) = ₹26,10,500.

Production defects cost ₹24,00,000, which is about 91.9% of the total recorded fixing cost. A production defect costs ₹40,000, while a requirements defect costs only ₹500. If the 60 production defects had been detected during testing, the estimated cost would have been 60 × ₹8,000 = ₹4,80,000. The possible difference is ₹19,20,000.

This is only an estimate. It assumes the defects could have been detected during testing and that the stated fixing costs are accurate. It does not include review costs, patient impact or other indirect costs. The assignment does not provide actual review efficiency, so the exact saving from reviews cannot be known. However, requirement and design reviews would likely reduce late defects.

Therefore, treating testing as a continuous process would help find defects earlier, reduce repair costs and improve patient confidence.

TASK 4 – APPLY AND CRITIQUE BASIC DEFINITIONS

Term	Meaning in MediQueue
Error	A human mistake, such as misunderstanding the GST rule or making an incorrect coding decision.
Fault / defect	A flaw in the requirement, design, code or test case that can cause incorrect behaviour.
Failure	The incorrect behaviour seen when a fault occurs, such as the wrong fee being charged.
Test case	A set of test steps, inputs and expected results used to check a feature.
Test oracle	The trusted source used to decide what the correct result should be, such as the approved hospital billing rule.
Test bed	The hardware, software, data and tools used to run tests, such as a test version of MediQueue with sample patients and doctors.

In I3, the error was misunderstanding the GST rule. The fault was incorrect fee calculation or display logic. The failure was that the page showed ₹500 but ₹550 was charged.

In I5, the error was misunderstanding how to calculate the patient's age on the appointment date. The fault was incorrect age-boundary logic. The failure was that a patient turning 60 did not receive senior-citizen priority.

In I6, the error was assuming that two doctors would not save prescriptions at the same time. The fault was unsafe handling of shared patient data. The failure was that one patient's prescription PDF showed another patient's details.

I8 happened because the tester used an incorrect expected refund value. The test was marked Pass even though the expected result was wrong. A common glossary is necessary so that everyone understands terms such as error, defect, failure, test case, test oracle, test bed, severity and priority. Correct terminology helps the team identify the real cause of problems.

TASK 5 – EVALUATE THE APPLICATION OF TESTING PRINCIPLES

The developers' claim that testing everything and finding no errors proves that the software is correct is wrong. The following testing principles apply to MediQueue.

1. Testing shows the presence of defects, not their absence. The successful demonstration did not prove that MediQueue was defect-free. Sixty defects were reported after release.

2. Exhaustive testing is impossible. It is not practical to test every patient name, booking time, age, network condition and possible input. The team should select important cases based on risk.

3. Early testing saves time and cost. Requirement and design reviews could have found problems such as the missing Sunday rule (I2) and GST misunderstanding (I3) before release. Production defects cost much more to fix.

4. Defects are often grouped in certain areas. Token allocation and shared prescription data caused serious problems in I1 and I6. These areas needed more testing.

5. Testing depends on context. A hospital application handles patient information, appointments and payments. It needs privacy, accuracy, performance and reliability testing.

6. Absence-of-error fallacy means that software may follow its written requirements but still fail users if the requirements are incomplete. I2 occurred because the Sunday closure rule was not written.

Early testing throughout the development life cycle would have had the greatest effect because it could identify unclear requirements and design problems before they became expensive production defects.

TASK 6 – EVALUATE THE TESTER'S ROLE

An independent tester would have improved MediQueue's quality because the tester could check the application from a patient's and hospital employee's point of view. However, one tester cannot guarantee that every defect will be found.

A tester should review requirements, prepare test cases, check designs, test the application, record defects, retest fixes and report whether the application is ready for release. During requirements review, the tester could clarify Sunday working days and GST rules. During system testing, the tester could check payments, reminders and performance.

A tester could most likely have detected I2 by reviewing requirements, I3 by checking the fee calculation, I4 by testing long names, I5 by testing the age boundary, I7 by checking the manual and I8 by reviewing the expected refund value. I1 and I6 would need planned concurrency tests. I9 would need integration tests and monitoring.

Developers testing their own code understand how it works and can find simple mistakes quickly. However, they may overlook problems because they use the same assumptions while developing and testing. An independent tester gives a fresh point of view.

A healthcare tester should have attention to detail, knowledge of hospital activities, technical testing skills, communication skills and respect for patient privacy. Therefore, an independent tester would have reduced the number of defects reaching patients.

TASK 7 – TRACE THE ORIGINS OF DEFECTS

Incident	Main origin	Reason
I1	Design	The token allocation design did not safely handle two bookings at the

		same time.
I2	Requirements	The Sunday closure rule was not written in the requirements.
I3	Requirements	The GST business rule was misunderstood.
I4	Coding	The application did not correctly handle names longer than 40 characters.
I5	Coding	The age calculation did not correctly handle a patient turning 60 on the appointment date.
I6	Design	Shared patient data was not safely handled when two doctors saved at once.
I7	Documentation	The manual mentioned a menu that did not exist.
I8	Testing	The tester used an incorrect expected refund value.
I9	Environment / external interface	The SMS service stopped working after the provider changed its API key.
I10	Requirements	A suitable performance requirement or response-time target was not defined.

Some incidents may have more than one origin. For example, I4 may also be related to a missing requirement about maximum name length. I9 may involve both external provider changes and weak application configuration handling. The main origin is selected based on the strongest evidence in the case.

The overall pattern shows that requirements and design need more attention. The defect data reports 30 defects from requirements and 22 from design, making 52 defects in total. The hospital should introduce requirement reviews and design inspections before development begins.

TASK 8 – EVALUATE DEFECT CLASSES AND SEVERITY

Defects can be classified as requirements defects, design defects, coding defects, testing defects, documentation defects and environment or external-interface defects. Severity describes the impact of a defect, while priority describes how quickly it should be fixed.

Incident	Defect class	Severity	Priority	Reason
I1	Design – concurrency	High	High	Two patients may receive the same token, affecting the appointment queue.
I2	Requirements – missing rule	Medium	High	Patients may book on a day when the OPD is closed.
I3	Requirements – billing rule	High	Critical	The patient is charged more than the

				amount displayed.
I4	Coding – input handling	High	Medium	The application crashes for long patient names.
I5	Coding – boundary condition	Medium	Medium	Eligible senior citizens may not receive priority.
I6	Design – shared data	Critical	Critical	One patient's prescription may show another patient's details.
I7	Documentation – incorrect instruction	Low	High	The manual gives an instruction for a menu that does not exist.
I8	Testing – incorrect expected result	High	High	A wrong test oracle may allow a refund defect to remain undetected.
I9	External	Medium	High	SMS

	interface / environment			reminders stop when the provider changes its API key.
I10	Performance requirement / design	Medium	Medium	The doctor's schedule screen takes 25 seconds to load.

Severity and priority can be different. I7 has low severity because it does not directly affect patient records or payments, but it can have high priority because correcting the manual is easy. I1 has high severity because duplicate tokens affect patients, but its priority depends on how often it occurs and whether staff have a temporary solution.

The hospital should first target requirements defects because 30 defects were introduced in that phase and only two were detected there. Better requirements can prevent problems before design and coding. Design defects involving patient data, such as I6, must also receive urgent attention because of their serious privacy impact.

TASK 9 – DESIGN AND APPRAISE A DEFECT REPOSITORY

A defect repository is a central place where the team records and tracks software problems. It should contain the following information.

Field	**Purpose**
Defect ID and title	Provides a unique number and short description for each defect.
Date and reporter	Shows when the defect was found and who reported it.

Module and version	Identifies the affected part of MediQueue and the software version.
Steps to reproduce	Allows another team member to repeat the problem.
Expected and actual result	Shows what should happen and what actually happened.
Defect class and origin	Helps identify whether the problem began in requirements, design, coding, testing or another area.
Severity and priority	Helps decide the impact and order of fixing defects.
Assigned person and status	Shows who is responsible and whether the defect is open, fixed or closed.
Fix details and retest result	Records the correction and confirms that it works.
Root cause and prevention action	Helps the team learn from the defect and avoid similar problems.

The repository helps the team analyse defects by grouping them according to type, phase and severity. It supports process improvement by showing repeated problems and helping the team prepare better reviews and test cases. E-mails and spreadsheets can contain duplicate or incomplete information and make it difficult to track the history of a defect. A central repository provides better tracking and reporting. Patient information should be protected and test data should be anonymised.

TASK 10 – EVALUATE DEFECT EXAMPLES AND PREVENTION STRATEGIES

I selected three incidents from different defect classes: I2, I6 and I8.

I2 – Requirements defect: The Sunday closure rule was not included in the requirements. The process change should be a requirement review with outpatient department staff before development. This needs some meeting time but can prevent invalid bookings and later corrections.

I6 – Design defect: The system did not safely handle two doctors saving prescriptions at the same time. The process change should be a design inspection for shared data and concurrency before coding. It requires technical review and testing effort, but it can prevent a serious patient privacy problem.

I8 – Testing defect: The refund test used an incorrect expected result. The process change should be an independent review of test cases and expected values against approved hospital rules. This adds a small review effort and helps prevent false Pass results.

Requirement review gives the best overall value because it can prevent many defects before coding. The defect data shows that 30 defects were introduced during requirements. However, the design inspection for I6 is also essential because patient privacy must be protected. Exact savings cannot be calculated because the assignment does not provide the cost or effectiveness of each prevention strategy.

TASK 11 – RECOMMEND AND DEFEND A QUALITY IMPROVEMENT PLAN

MediQueue needs a planned quality improvement process because many defects reached patients after release. The following changes should be introduced in priority order.

1. Review requirements with hospital staff to confirm appointment rules, fees, age limits and working days.

2. Conduct design reviews for token allocation, shared patient data and external services.

3. Assign an independent tester to prepare test cases, test important workflows and report defects.

4. Use coding standards, peer reviews and unit tests for input handling and boundary conditions.

5. Maintain a central defect repository with defect details, severity, priority, root cause, fix and retest results.

6. Perform integration, system, performance and regression testing before every release.

7. Approve releases only when critical defects are fixed and important tests have passed.

This plan should reduce defects reaching production because problems will be checked during requirements, design, coding and testing. It should also reduce fixing costs because production defects cost ₹40,000 each, compared with ₹500 for requirements defects.

The plan has some limitations. Reviews and testing require time, and the first few releases may take longer. The five developers may have limited time and testing experience. The hospital can manage this by starting with high-risk features such as appointments, payments and prescriptions, and by using short checklists and reusable test cases.

The plan is practical for a small team if quality is treated as a shared responsibility. Developers can perform peer reviews and unit tests, while an independent tester focuses on system-level and high-risk testing. Management should review defect trends after each release and improve the process based on actual results.

TASK 12 – RE-EVALUATE AND ADAPT THE QUALITY PROCESS

A. Impact Analysis

The situation has changed because MediQueue must now release a new version every two weeks instead of every six months. It must also connect to the national digital health ID system, which is maintained by an external agency. Patient health records will be shared between hospitals.

Frequent releases reduce the time available for manual testing. The team must test continuously and repeat important tests after every change. Sharing health records increases the risk of privacy problems and incorrect patient information. Therefore, data protection, access control and accurate record matching become very important.

The tester's role will change from testing mainly before release to working throughout development. The tester must help prepare test cases early, check external connections, maintain regression tests and verify privacy-related features.

B. Evaluation of the Original Plan

The original plan is still useful because requirement reviews, design inspections, independent testing and a defect repository are necessary. However, the same manual process may not fit into a two-week release cycle.

The external health ID system introduces new possible defect origins, such as API changes, incorrect data mapping, authentication problems, service unavailability and differences between test and production environments. The defect repository should record the external system version, API details, responsible organisation, defect status and related agency ticket number.

C. Adaptation or Redesign

My decision is to MODIFY the original quality process. Keeping it unchanged would not suit frequent releases, while completely redesigning it would remove useful practices that already address MediQueue's problems.

The revised process should include the following steps:

- Review requirements and acceptance criteria with hospital staff before each development cycle.
- Review designs that involve patient identity, shared records and external connections.
- Perform coding reviews and unit testing during development.
- Run automated regression tests for booking, tokens, payments, prescriptions and reminders.
- Perform integration and contract testing with the health ID system's test environment.
- Conduct privacy, access-control, performance and security checks for high-risk changes.
- Use a release checklist, monitoring and a rollback plan before deployment.
- Review defects after each release and update test cases to prevent repeated problems.

The tester will work with developers throughout the two-week cycle, maintain reusable tests, check the external interface and report release risks. Defects from the external system should

be recorded separately from MediQueue coding defects, with clear ownership and links to the external agency's issue records.

D. Evidence-Based Defence

The defect data shows that 60 defects were reported after release and the production fixing cost was ₹24,00,000. This supports keeping early reviews and strong release checks. Requirements and design introduced 52 defects, so these activities must remain part of the process.

The two-week release schedule means that manual testing alone may take too much time. Automated regression and contract tests will help the small team repeat important checks quickly. Sharing health records increases the impact of privacy and data-mixing defects such as I6, so privacy and access-control testing must receive high priority.

Therefore, modifying the original process is the best decision. It keeps useful quality practices and adds automation, external-interface testing, privacy checks and continuous testing. The team should review the results after several releases and adjust the process based on actual defect and effort data.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	5
Task 2 – Evaluate the Role of Process in Software Quality	2
Task 3 – Evaluate Testing as a Process	6
Task 4 – Apply and Critique Basic Definitions	4
Task 5 – Evaluate the Application of Testing Principles	3
Task 6 – Evaluate the Tester's Role	3
Task 7 – Trace the Origins of Defects	4
Task 8 – Evaluate Defect Classes and Severity	4
Task 9 – Design and Appraise a Defect Repository	3
Task 10 – Evaluate Defect Examples and Prevention Strategies	4
Task 11 – Recommend and Defend a Quality Improvement Plan	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50